

Sistemas de Inteligencia Artificial

Repaso

Centro de Inteligencia Computacional

2025

Autoencoder lineal

Descomposición Espectral

Repasando la clase de Aprendizaje No supervisado y PCA, si una matriz X es invertible y sus autovalores son reales, entonces la matriz puede ser diagonalizable y factorizarse

$$X = ELE^T \quad (1)$$

Donde L es la matriz diagonal formada por los autovalores y E es la matriz de filas formada por sus autovectores.

Autoencoder lineal

Descomposición en valores singulares

Para el caso de matrices no cuadradas, existe una operación similar pero más general que es la descomposición en valores singulares, SVD. Dada una matriz X de $n \times d$,

$$SVD(X) = \tilde{Z}\Sigma V^T \quad (2)$$

Donde $\tilde{Z}$ son los vectores columnas singulares de izquierda, Σ es una matriz diagonal positiva y V son los vectores singulares derechos. El cálculo de SVD se realiza minimizando

$$\|X - \tilde{Z}\Sigma V^T\|. \quad (3)$$

Regularización

Regularizar implica agregar una condición extra R sobre la función objetivo L que limita el espacio de búsqueda de las soluciones.

$$J = L + \lambda R = \min_{\phi} \frac{1}{N} \|Y - X\phi\| \underbrace{(+\lambda \|f(\phi)\|)}_{\text{Término Regularizador}} \quad (4)$$

Desalienta las soluciones complejas o extremas en un problema de optimización. El parámetro de regularización λ se usa justamente para ajustar la importancia que se le da a al término regularizador (Tikhonov Regularization).

Repaso: Aproximación de una pdf

¿Si tengo un generador de números pseudoaleatorios que se corresponde con una distribución de probabilidad uniforme, y tengo una p.d.f., ¿Cómo puedo usar el generador para generar números aleatorios que se corresponden con la p.d.f. ?

Inverse Sampling Theorem

$$Y \sim U[0, 1], F \text{ invertible y } F = CDF(pdf()), \\ X = F^{-1}(Y) \text{ tiene } CDF(X) = F$$

$$Y \sim U[0, 1], F = CDF(pdf()), X = \inf_x \{x : F(x) \geq Y\}$$

Aproximación de una pdf

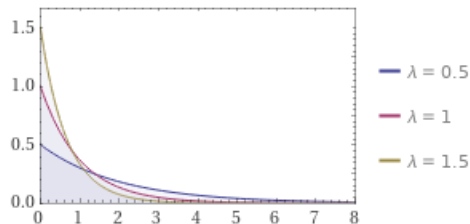
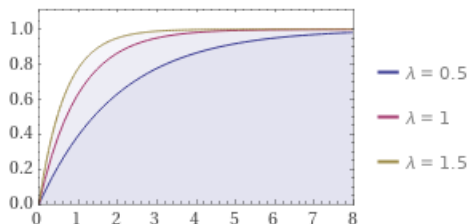
Eligen un valor de $Y \sim U[0, 1]$, se fijan que les da en $X = F^{-1}(Y)$. La distribución de esos X va a corresponder con la *pdf* en cuestión.

Inverse Sampling Theorem

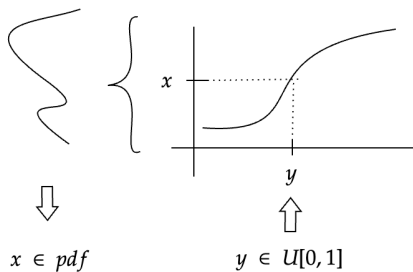
$Y \sim U[0, 1]$, F invertible y $F = CDF(pdf())$,
 $X = F^{-1}(Y)$ tiene $CDF(X) = F$

$Y \sim U[0, 1]$, $F = CDF(pdf())$, $X = \inf_x \{x : F(x) \geq Y\}$

Aproximación de una p.d.f.



Aproximación de una p.d.f.



$$F = CDF(pdf)$$

$$\downarrow$$

$$\leftarrow F^{-1}$$

Aproximación de una p.d.f.

Teorema Central del Limite

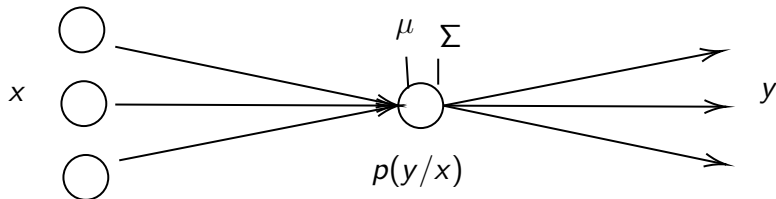
Para el caso de Gaussianas, se puede utilizar el TCL, obtener un número grande de $y \in U[0, 1]$ y con eso generar correspondiente $N(\frac{n}{2}, \frac{n}{12})$.

Stochastic Feedforward Neural Networks [1]

Ahora, ¿ Cómo podríamos hacer eso con una red neuronal ?

Una red de una capa oculta es un aproximador universal[1] puede representar cualquier función computable (Arbitrary Continuous Functions, Cybenki's Theorem). Entre otras cosas, puede representar una función estocástica, regida por una p.d.f.

Stochastic Feedforward Neural Networks



Reparametrization Trick

Ahora, ¿ Cómo se construye la capa estocástica ?

La clave está en el **reparametrization trick** haciendo que el valor de z se calcula en función de $\bar{\mu}$ y de $\bar{\Sigma}$.

$$z = h(\bar{X}) = \epsilon \odot \bar{\Sigma}(\bar{X}) + \bar{\mu}(\bar{X}) \quad (5)$$

Colab - Reparametrization Trick

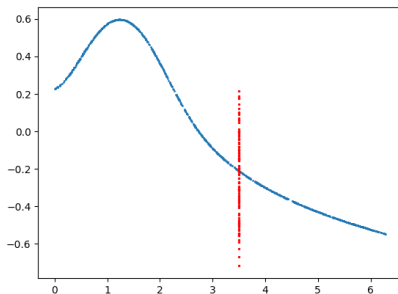
`https://colab.research.google.com/drive/
1N8HMTNgCX0flDEwbx9Um6hD1D1vS1Ha?usp=sharing`

Stochastic Functions[2]

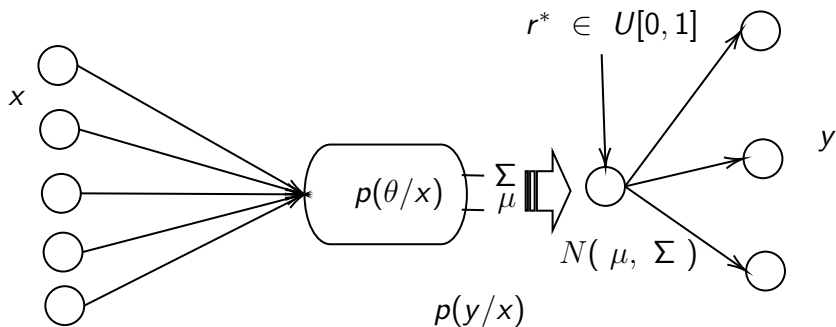
Una función donde la asignación de x a y es estocástica, regida por una p.d.f. condicional.

$$f(x) = L(x) + \epsilon(x)$$

$y = f(x)$ tal que, dado x , $y = f(x)$ según $p(y/x)$.



Stochastic Feedforward Neural Networks



Repaso: Teoría de la Información

Variable aleatoria: $X = x_i$

Información: $I = -\log p(x_i)$

Entropía: Más o menos, cuanta info tienen todos los estados posibles que puede tomar x . Promedio ponderado.

$$H = \mathbb{E}_{p(x_i)} I(x_i) = -\sum p(x_i) \log p(x_i)$$

Información Mutua:

$$I(x, y) = H(y) - H(y/x) = \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

$H(x) = I(x, x)$, y se tiene que,

- $I = 0$ implica que $p(x) = 1$ Certeza !
- $I > 0$ algo de información
- $I(x_k) > I(x_i) \Rightarrow P_k < P_i$, lo más raro (menos probable) aporta más información.

Repaso: Teoría de la Información

La entropía es mínima, cero, cuando es una situación de total certeza (teniendo en cuenta que $\lim_{p \rightarrow 0^+} p \log p = 0$), y es máxima, cuando todos los eventos son equiprobables (probabilidad uniforme).

$$0 \leq H(x) \leq \log(2k + 1)$$

Entropía condicional: $H(x/y) = H(x, y) - H(y)$

Entropía conjunta: $H(x, y) = \sum_{i=1}^m \sum_{j=1}^m p(x_i, y_j) \log \frac{1}{p(x_i, y_j)}$

Repaso: Teoría de la Información

Cross Entropy

$$H_p(q) = - \sum_{k=1} q(y_k) \log p(y_k)$$

Binary Cross Entropy

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N \{(y_i) \log p(y_i) + (1 - y_i) \log(1 - p(y_i))\}$$

$q(y_k) = y_i = 0|1$, y representa el label, $p(y_i)$, $1..N$ es la probabilidad asignada a ese label para la muestra i . Binary Cross Entropy sirve como una función para estimar cuál es la efectividad de un clasificador que asigna el label y a cada elemento de $1..N$ con probabilidad $p(y_k)$. Un clasificador perfecto que estima y_i con probabilidad 1, genera un valor de binary cross entropy de cero.

KL: Kullback-Leibler Divergencia ó Entropía relativa

Es una "norma" (en rigor no) que permite ver la distancia entre distribuciones de probabilidad, i.e. si tiende a cero implica que las distribuciones son parecidas. Una medida de similaridad.

KL

$$KL(q||p) = - \sum_{x_i} q(x) \log \frac{q(x)}{p(x)}$$

donde $p(x)$ es la referencia a la que se compara $q(x)$.

KL: Kullback-Leibler Divergencia ó Entropía relativa

$$KL(q||p) = - \sum_{x_i} q(x) \log \frac{q(x)}{p(x)}$$

Propiedades:

- $KL(q||p) \neq KL(p||q)$
- $KL(q||q) = 0$
- $KL(q||p) \geq 0$
- $KL(q||p) = H_p(q) - H(q) \geq 0$

Referencias I

- [1] Charlie Tang and Russ R Salakhutdinov. Learning stochastic feedforward neural networks. *Advances in Neural Information Processing Systems*, 26, 2013.
- [2] Eric W. Weisstein. *Stochastic Function*, 2023 (accessed November 17, 2023).
<https://mathworld.wolfram.com/StochasticFunction.html>.